DOP853: Numerical Method and Adaptive Execution

1 Differential equation and numerical method

Consider $z' = f(t, z)$, $z(t_0) = z_0$, in the project's packed physical layout. An explicit Runge–Kutta method of order eight, with embedded local-error estimation and a seventh-order dense interpolant. The implementation delegates the numerical tableau, local error estimator, trial rejection and interpolation polynomial to the installed SciPy `DOP853` solver. The adapter does not duplicate these algorithms.

A Runge–Kutta evaluation has stage equations

$$Y_i = z_n + h \sum_j a_{ij} f(t_n + c_j h, Y_j), \qquad z_{n+1} = z_n + h \sum_i b_i f(t_n + c_i h, Y_i).$$

The explicit tableau evaluates stages sequentially. No nonlinear system, Jacobian or LU factorization is required.

2 Error control and accepted intervals

The adaptive backend scales local error estimates using positive absolute and relative tolerances, with component scale based on $\mathrm{atol} + \mathrm{rtol} \max(|z_n|, |z_{n+1}|)$. It selects subsequent step sizes and can reject trial steps. `max_step` bounds accepted duration; `first_step=None` delegates initial selection to the backend. Requested tolerances bound neither global trajectory error nor roundoff.

One live solver is retained over the complete interval. Its state, stage work and any Jacobian/factorization reuse survive accepted steps. The adapter calls `solver.step()` until acceptance or failure; rejected trials never become observer records or separate accepted metric rows.

3 Dense output and energy extension

One retained SciPy explicit adaptive solver. Energy tracking is optional and passive. Eight-point Gauss quadrature integrates the passive derivative along each accepted dense interpolant. Partial-interval quadrature supplies off-grid energy samples. This diagnostic quadrature has no separate adaptive tolerance or rigorous error bound; refine the physical step and check the balance. The common state conventions are specified below.

4 Shared architecture and observation

The public class owns initialization, advance, observation and history export. Inherited `new_run` constructs a fresh instance of the same class and calls `initialize`, which creates its live solver. There is no separate method context or record of callbacks. All 13 methods inherit the same integration entry point and use the common global coordinator.

`ScipyAdaptiveController` exposes accepted intervals from the retained numerical session. The common collector stores requested samples and small accepted-step statistics; it does not retain interpolants or events by default.

An `AdaptiveIntegrationStep` records times and copied states. It also exposes work counters and `dense_state`. It does not claim a fixed-duration candidate-state map: adaptive decisions depend on numerical history. `AdaptiveTrajectoryObserver` explicitly retains continuous output outside the integrator. The observer and any persistence belong to the caller.

Function evaluations, Jacobian evaluations and LU decompositions are counted per accepted interval. Initialization work belongs to the first row; rejected trial work belongs to the acceptance that follows it. Interpolation needed only for an observer is excluded from main counters unless continuous output was explicitly requested. The ordinary method operation `advance` accepts one step on the retained solver, taking an upper step bound. The controller reports the actual accepted endpoint. A run instance belongs to one execution; repeating `simulate` creates a fresh instance and solver.

5 Implementation and verification

The adapter is in `src/methods/adaptive/scipy.py`; shared coordination and collection are in `src/execution/_host.py`. The current state and execution contracts are documented in its `simulation/` directory. Tests compare sampled states and work counters with the same SciPy solver, check a known non-autonomous rotation, and verify optional energy, dense retention, repeated runs, observation isolation and failure handling. Scientific reference trajectories still require refinement and an independent audit; arithmetic precision and integration error remain separate.

6 Physical accepted states and passive energy tracking

For N particles with d physical components, every accepted snapshot contains $Z \in \mathbb{R}^{dN}$, one scalar time t, and optional separate physical momentum $\kappa \in \mathbb{R}^N$. Thus GC accepted vectors have $2N$ components and FC vectors have $4N$, whether energy is tracked or not. A doubled method prepares temporary $(u, v) \in \mathbb{R}^{2dN}$ copies, allows them to separate, and projects to one physical result before acceptance. The representation stores only the physical size and never owns a clock, momentum, physical coordinate declaration, dynamics or solver.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, Z), \qquad \kappa(t_0) = 0, \qquad \Delta K(t) = H(t, Z(t)) + \kappa(t) - H(t_0, Z_0).$$

GC split-map contributions are summed in temporary work and divided by two to obtain physical momentum. Tracking leaves physical stages, nonlinear stopping scales and adaptive acceptance unchanged. An autonomous Hamiltonian supplies an explicit zero momentum derivative.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired. Dynamics declare component order and position/velocity roles through immutable structural data; initial conditions and results retain those data independently. Methods own maps, projection equations and momentum updates. Generic copy operations live in `src/formulations/state.py`.

Controllers and compiled JAX loops carry physical values and momentum separately. Observers receive physical states. Energy histories in `States.diagnostics` retain their existing labels; `extended_time` has shape (N, n_{samples}) and is broadcast from scalar sample times. Fixed shadow samples leave accepted states and counters unchanged. Adaptive quadrature uses accepted dense output and physical-only error control.

7 Optional JAX execution

The optional JAX execution choice preserves the same adaptive method and SciPy controller, including accepted/rejected step control and dense output. JAX evaluates particle field and passive-energy batches on a selected CPU or GPU, returning them to the host solver. This is explicitly reported as hybrid execution; it does not move the adaptive solver or Radau linear algebra onto the GPU. Physical error norms and passive energy decoupling are unchanged.

Both execution routes use double precision and return the same physical solution layout. Floating-point reassociation can produce round-off differences without changing the designed order or geometric properties. First-call compilation is separate from warmed integration timings, which include transfers and output construction. The sibling runtime document and shared execution contract specify supported callbacks and exact device scope.